



Vivekanand Education Society's BUSINESS SCHOOL

SUBMITTED BY

Name of the Student:

72 - Harshit Saraf

74 - Jishnu Pramod

78 - Kiran Gaikwad

87 - Poorva Palkar

Experiential Learning for Course: Setorial Analysis of Indian District and States.

Faculty Name: **Dr. Pranjal Muley**

INDUSTRY MENTOR

“Name”

PGDM 2024-2026

Academic year: 24-25

Vivekanand Education Society's Business School

Table of Contents

Sr. no.	Title	Page No.
1	Brief Introduction of the company	7
2	Brief of the Job Assigned	8
3	Targets Assigned	9
4	Targets Achieved	9
5	Results	10 - 23
6	Learnings	24
7	Limitations	24 - 26
8	Conclusions	26
9	Dashboard	26

Introduction of the Company



Strategic Execution Consultants Pvt. Ltd.(SEC), based in Bangalore, Karnataka, specializes in consulting services that support business transformation, corporate finance, strategic planning, and digital innovation. Founded with a mission to redefine strategic success, SEC provides tailored solutions across multiple sectors including financial services, healthcare, education, and retail. The firm's offerings span from business strategy and go-to-market planning to cutting-edge technology services like generative AI and data intelligence, aimed at optimizing business operations.

SEC's expertise is evident in its commitment to delivering value-driven solutions that improve operational efficiency, compliance, and customer engagement. The company has established a reputation for helping clients navigate complex challenges, whether through risk management for financial institutions, digital transformation strategies for improved customer journeys, or data-driven decision-making in education and healthcare.

The company operates out of its office at HAL 3rd Stage, Kodihalli, Bangalore, with services tailored to the unique demands of each industry, aiming for sustainable growth and market competitiveness.

Brief of the Job assigned

The assigned task was part of a project titled "**District-Wise Sectoral Analysis of Indian States.**"

The primary objective was to analyze a comprehensive dataset, which captures economic contributions from the primary, secondary, and tertiary sectors of various districts in India across different years.

The dataset consists of economic metrics at both constant and current prices, alongside sectoral shares in district-level GDP.

The task required the use of SQL to query, filter, and extract meaningful insights, such as identifying growth trends, comparing sectoral performance, and calculating year-on-year changes in economic contributions.

Additionally, the task involved presenting these insights using data visualization tools for clarity.

Targets Assigned

The following targets were set for this project:

- Gain a thorough understanding of the dataset's structure, which includes district codes, state names, sector contributions, and economic shares.
- Write SQL queries to retrieve data for specific states and districts, focusing on total contributions, sectoral breakdowns, and growth trends.
- Perform aggregation to find total sectoral contributions across all districts for a particular year and join data for state-level analysis.
- Analyze sector-wise performance by calculating percentage shares and identifying districts with significant contributions.
- Visualize the findings to make insights more accessible and understandable

Targets Achieved

Through the execution of SQL queries, the following targets were successfully met:

- Data for specific states, such as Chhattisgarh, was retrieved to analyze trends over time.
- SQL queries were developed to filter districts with high economic contributions, aggregate sectoral data, and join district and state tables for a comprehensive analysis.
- Year-on-year growth for key sectors, particularly the secondary and tertiary sectors, was calculated, showing how different sectors evolved in various districts.
- A trend analysis was conducted, revealing the districts with the highest growth rates in the tertiary sector over a five-year period.
- Grouped data was used to compute total contributions by the secondary sector for each state, providing state-level economic insights.
- Visual representations were created using Power BI to depict growth trends and compare sectoral contributions across districts and states.

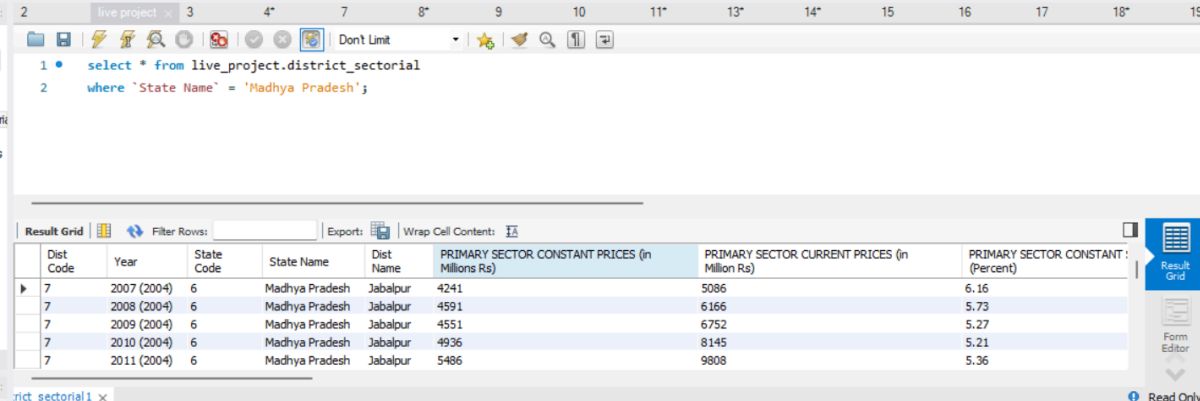
RESULTS

1. Basic Query:

- Retrieve all data for a specific state, e.g., "Chhattisgarh."

select * from live_project.district_sectorial

where `State Name` = 'Madhya Pradesh';



The screenshot shows a SQL query editor with a query window and a results grid. The query is:

```
1 select * from live_project.district_sectorial
2 where `State Name` = 'Madhya Pradesh';
```

The results grid displays the following data:

Dist Code	Year	State Code	State Name	Dist Name	PRIMARY SECTOR CONSTANT PRICES (in Millions Rs)	PRIMARY SECTOR CURRENT PRICES (in Million Rs)	PRIMARY SECTOR CONSTANT : (Percent)
7	2007 (2004)	6	Madhya Pradesh	Jabalpur	4241	5086	6.16
7	2008 (2004)	6	Madhya Pradesh	Jabalpur	4591	6166	5.73
7	2009 (2004)	6	Madhya Pradesh	Jabalpur	4551	6752	5.27
7	2010 (2004)	6	Madhya Pradesh	Jabalpur	4936	8145	5.21
7	2011 (2004)	6	Madhya Pradesh	Jabalpur	5486	9808	5.36

2. Filtering:

- Show all the districts in a given state that had a total contribution

select `State Name`, `Total Current Prices (in Millions Rs)`, `Dist Name`, `Year` from district_sectorial where `State Name` = "Madhya Pradesh" and `Total Current Prices (in Millions Rs)` > 100000 and `Year` = 2010;

SQL Query:

```
1 • select `State Name`,`Total Current Prices (in Millions Rs)`,`Dist Name`,`Year` from district_sectorial where `State Name` = "Madhya Pradesh"
2 | and `Total Current Prices (in Millions Rs)` > 100000 and `Year` = 2010;
```

Result Grid:

State Name	Total Current Prices (in Millions Rs)	Dist Name	Year
Madhya Pradesh	139334	Jabalpur	2010 (200-4)
Madhya Pradesh	109287	Gwalior	2010 (200-4)
Madhya Pradesh	314004	Indore	2010 (200-4)

3. Aggregation:

- Find the total contribution of the primary Join Operation: Write a query to join district data with state codes and names to display state-wise analysis. sector across all districts for a given year.

select `Dist Name`,`Year`, sum(PRIMARY SECTOR CONSTANT PRICES (in Millions Rs)+ `PRIMARY SECTOR CURRENT PRICES (in Million Rs)`) as total_primary_sector

from district_sectorial

group by `Dist Name`,`Year`;

SQL Query:

```
1 • select `Dist Name`,`Year`, sum( PRIMARY SECTOR CONSTANT PRICES (in Millions Rs)+ `PRIMARY SECTOR CURRENT PRICES (in Million Rs)` ) as total_primary_se
2 | from district_sectorial
3 | group by `Dist Name`,`Year`;
```

Result Grid:

Dist Name	Year	total_primary_sector
Durg	2007 (200-4)	33089
Durg	2008 (200-4)	30752
Durg	2009 (200-4)	36065
Durg	2010 (200-4)	60770
Durg	2011 (200-4)	65377
Durg	2012 (200-4)	76010

4. Join Operation:

- Write a query to join district data with state codes and names to display state-wise analysis.

```
select a.`State Code`,b.`State Name`,sum(a.`PRIMARY SECTOR CONSTANT
PRICES (in Millions Rs)`) as 'total primary constant price',sum(a.`SECONDARY
SECTOR CONSTANT PRICES (Millions in Rs)`) as 'total primary constant price',
```

```
sum(a.`TERTIARY SECTOR CONSTANT PRICES (Millions in Rs)`) as 'total
primary constant price'
```

```
from district_sectorial a
```

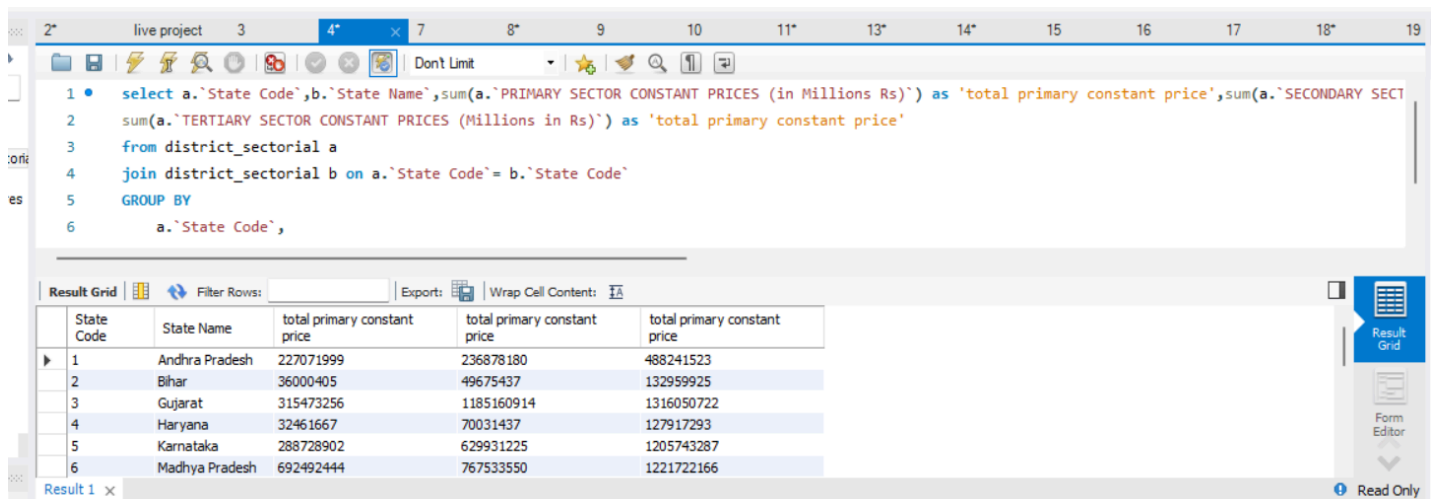
```
join district_sectorial b on a.`State Code`= b.`State Code`
```

```
GROUP BY
```

```
a.`State Code`,
```

```
b.`State Name`
```

```
order by a.`State Code` asc;
```



The screenshot shows a SQL IDE interface with a query editor and a results grid. The query is as follows:

```
1 select a.`State Code`,b.`State Name`,sum(a.`PRIMARY SECTOR CONSTANT PRICES (in Millions Rs)`) as 'total primary constant price',sum(a.`SECONDARY SECT
2 sum(a.`TERTIARY SECTOR CONSTANT PRICES (Millions in Rs)`) as 'total primary constant price'
3 from district_sectorial a
4 join district_sectorial b on a.`State Code`= b.`State Code`
5 GROUP BY
6 a.`State Code`,
```

The results grid displays the following data:

State Code	State Name	total primary constant price	total primary constant price	total primary constant price
1	Andhra Pradesh	227071999	236878180	488241523
2	Bihar	36000405	49675437	132959925
3	Gujarat	315473256	1185160914	1316050722
4	Haryana	32461667	70031437	127917293
5	Karnataka	288728902	629931225	1205743287
6	Madhya Pradesh	692492444	767533550	1221722166

5. Growth Calculation:

- Calculate the year-on-year growth of the secondary sector for a specific district.


```

select curr.`Year`, curr.`State Code`, curr.`Dist Name`, curr.`SECONDARY SECTOR
CURRENT PRICES (Millions in Rs)` AS `Current_Year_Price`,
prev.`SECONDARY SECTOR CURRENT PRICES (Millions in Rs)` as
`Previous_Year_Price`, ((curr.`SECONDARY SECTOR CURRENT PRICES
(Millions in Rs)` - prev.`SECONDARY SECTOR CURRENT PRICES (Millions in
Rs)`)/prev.`SECONDARY SECTOR CURRENT PRICES (Millions in Rs)`*100) AS
`Growth_Rate`

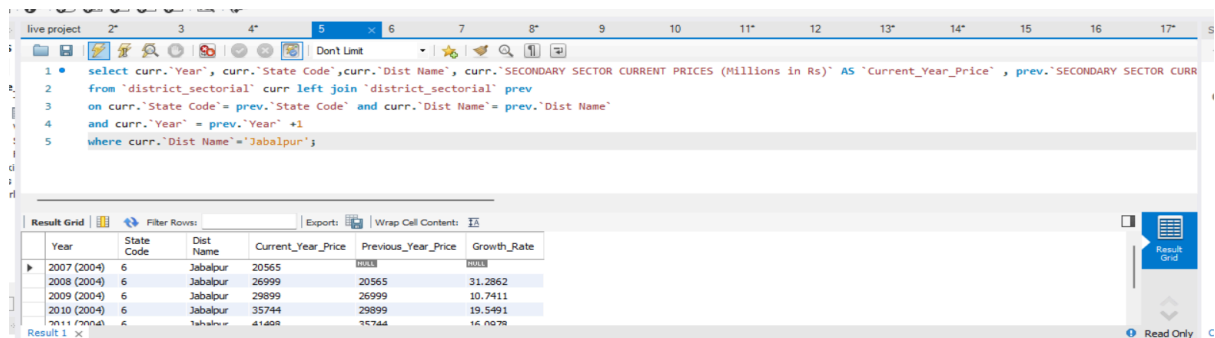
from `district_sectorial` curr left join `district_sectorial` prev

on curr.`State Code`= prev.`State Code` and curr.`Dist Name`= prev.`Dist Name`

and curr.`Year` = prev.`Year` +1

where curr.`Dist Name`='Jabalpur';

```



The screenshot shows a SQL query editor with a query window and a results grid. The query is a self-join on the 'district_sectorial' table to calculate the growth rate of secondary sector current prices for Jabalpur district over a five-year period. The results grid displays the following data:

Year	State Code	Dist Name	Current_Year_Price	Previous_Year_Price	Growth_Rate
2007 (2004)	6	Jabalpur	20565	18888	10.0000
2008 (2004)	6	Jabalpur	26999	20565	31.2862
2009 (2004)	6	Jabalpur	29899	26999	10.7411
2010 (2004)	6	Jabalpur	35744	29899	19.5491
2011 (2004)	6	Jabalpur	41408	35744	16.1078

6. Trend Analysis:

- Identify the district with the highest growth rate in the tertiary sector over a five-year period.

```

select curr.`Dist Name`, curr.`year` as `currentYear`, prev.`year` as
`previousYear`, max(((curr.`TERTIARY SECTOR CURRENT PRICES (Millions in
Rs)` - prev.`TERTIARY SECTOR CURRENT PRICES (Millions in
Rs)`)/prev.`TERTIARY SECTOR CURRENT PRICES (Millions in Rs)`*100)) AS
`Growth_Rate`

```

```

from `district_sectorial` curr join `district_sectorial` prev

on curr.`State Code`= prev.`State Code` and curr.`Dist Name`= prev.`Dist Name`
where curr.`Year`-prev.`Year`=5

group by curr.`Dist Name`, curr.`Year`, prev.`Year`

```

order by `Growth\_Rate` desc limit 1;

The screenshot shows a SQL query editor with the following query:

```
1 • select curr.`Dist Name`,curr.`year` as `currentYear`, prev.`year` as `previousYear`,max(((curr.`TERTIARY SECTOR CURRENT PRICES (Millions in Rs)`- prev.`TERTIARY SECT
2 from `district_sectorial` curr join `district_sectorial` prev
3 on curr.`State Code`= prev.`State Code` and curr.`Dist Name`= prev.`Dist Name` where curr.`Year`-prev.`Year`=5
4 group by curr.`Dist Name`, curr.`Year`, prev.`Year`
5 order by `Growth_Rate` desc limit 1;
```

The result grid shows the following data:

Dist Name	currentYear	previousYear	Growth_Rate
Gurgaon	2012 (2004)	2007 (2004)	234.5239

7. Group By:

- Group data by state to calculate the total contribution of the secondary sector in constant prices for each state.

SELECT `State Name`, SUM(`SECONDARY SECTOR CONSTANT PRICES (Millions in Rs)`) AS `Total Secondary Contribution`

FROM district_sectorial

GROUP BY `State Name`;

The screenshot shows a SQL query editor with the following query:

```
1 • SELECT `State Name`, SUM(`SECONDARY SECTOR CONSTANT PRICES (Millions in Rs)`) AS `Total Secondary Contribution`
2 FROM district_sectorial
3 GROUP BY `State Name`;
```

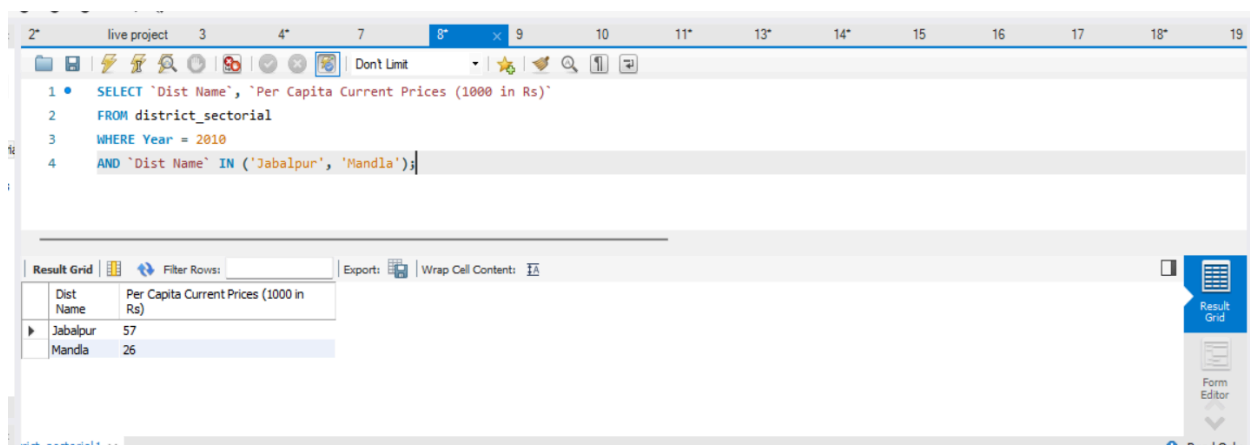
The result grid shows the following data:

State Name	Total Secondary Contribution
Chhattisgarh	1412106
Madhya Pradesh	2963450
Andhra Pradesh	3076340
Telangana	2703901
Karnataka	4736325
Tamil Nadu	5342359

8. Comparison:

- Compare the per capita current prices between two districts over the same year.

```
SELECT `Dist Name`, `Per Capita Current Prices (1000 in Rs)`
FROM district_sectorial
WHERE Year = 2010
AND `Dist Name` IN ('Jabalpur', 'Mandla');
```



The screenshot shows a SQL IDE interface with a query editor at the top and a result grid at the bottom. The query in the editor is:

```
1 • SELECT `Dist Name`, `Per Capita Current Prices (1000 in Rs)`
2 FROM district_sectorial
3 WHERE Year = 2010
4 AND `Dist Name` IN ('Jabalpur', 'Mandla');
```

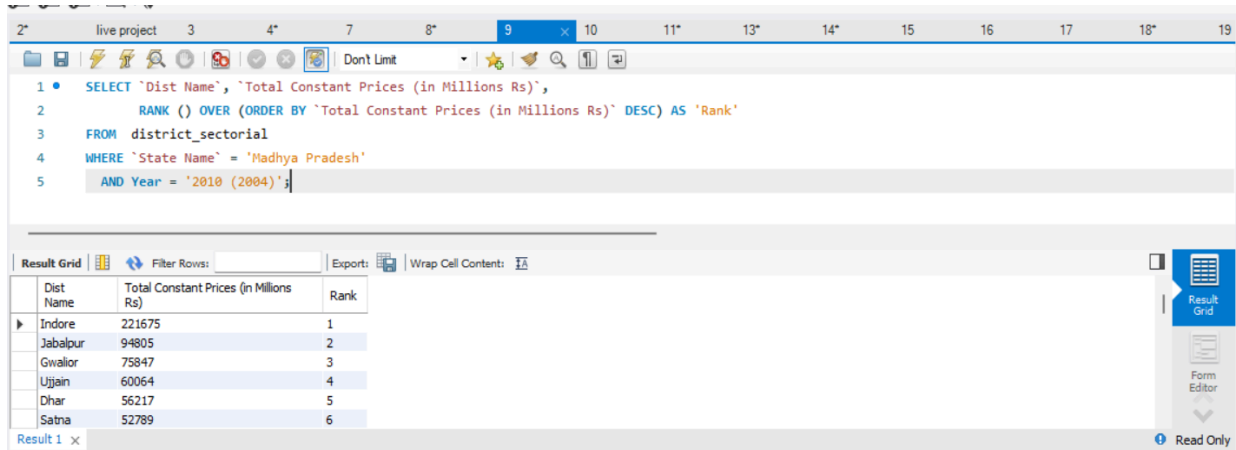
The result grid below the query shows the following data:

Dist Name	Per Capita Current Prices (1000 in Rs)
Jabalpur	57
Mandla	26

9. Ranking:

- Rank districts within a state based on their total constant prices for a given year.

```
SELECT `Dist Name`, `Total Constant Prices (in Millions Rs)`,
      RANK () OVER (ORDER BY `Total Constant Prices (in Millions Rs)` DESC) AS
`Rank`
FROM district_sectorial
WHERE `State Name` = 'Madhya Pradesh'
AND Year = '2010 (2004)';
```



```

1 • SELECT 'Dist Name', 'Total Constant Prices (in Millions Rs)',
2       RANK () OVER (ORDER BY 'Total Constant Prices (in Millions Rs)' DESC) AS 'Rank'
3 FROM district_sectorial
4 WHERE 'State Name' = 'Madhya Pradesh'
5       AND Year = '2010 (2004)';

```

Dist Name	Total Constant Prices (in Millions Rs)	Rank
Indore	221675	1
Jabalpur	94805	2
Gwalior	75847	3
Ujjain	60064	4
Dhar	56217	5
Satna	52789	6

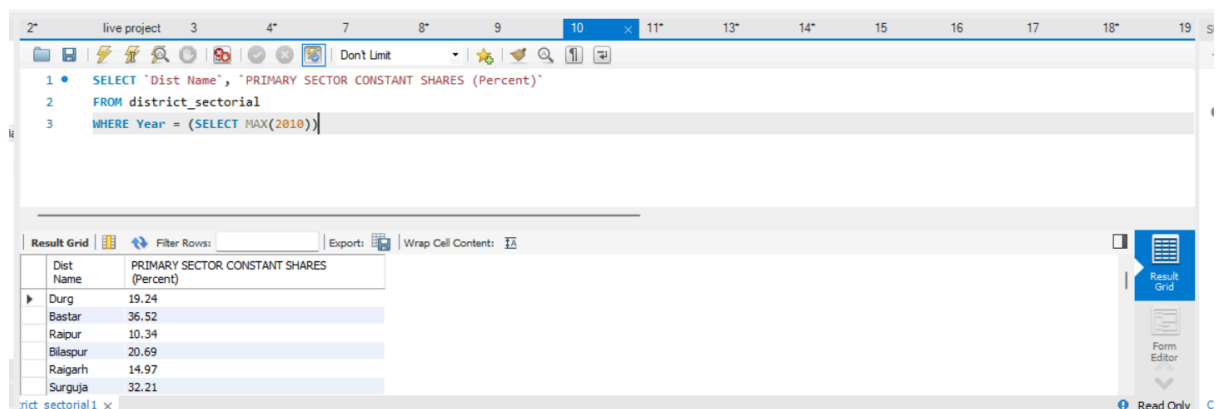
10. Filtering by Sector:

- Find the districts where the primary sector's share in constant prices is higher than 20% in the latest year.

SELECT 'Dist Name', 'PRIMARY SECTOR CONSTANT SHARES (Percent)'

FROM district_sectorial

WHERE Year = (SELECT MAX(2010))



```

1 • SELECT 'Dist Name', 'PRIMARY SECTOR CONSTANT SHARES (Percent)'
2 FROM district_sectorial
3 WHERE Year = (SELECT MAX(2010));

```

Dist Name	PRIMARY SECTOR CONSTANT SHARES (Percent)
Durg	19.24
Bastar	36.52
Raipur	10.34
Bilaspur	20.69
Raigarh	14.97
Surguja	32.21

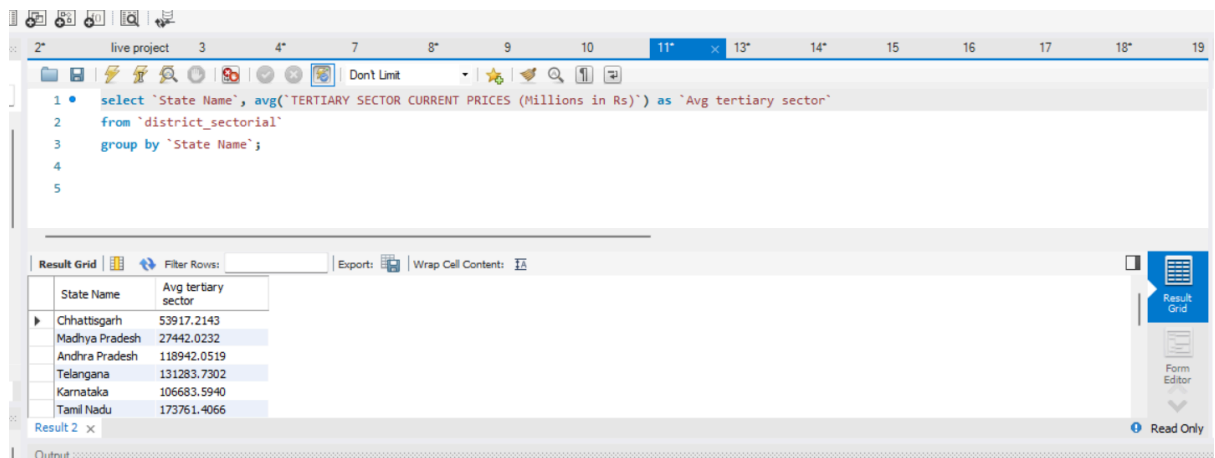
11. Sector Performance:

- For each state, calculate the average share of the tertiary sector (current prices) over the entire time period.

```
select `State Name`, avg(`TERTIARY SECTOR CURRENT PRICES (Millions in Rs)`) as `Avg tertiary sector`
```

```
from `district_sectorial`
```

```
group by `State Name`;
```



The screenshot shows a SQL IDE interface with a query editor and a results grid. The query in the editor is:

```
1 • select `State Name`, avg(`TERTIARY SECTOR CURRENT PRICES (Millions in Rs)`) as `Avg tertiary sector`
2   from `district_sectorial`
3   group by `State Name`;
4
5
```

The results grid displays the following data:

State Name	Avg tertiary sector
Chhattisgarh	53917.2143
Madhya Pradesh	27442.0232
Andhra Pradesh	118942.0519
Telangana	131283.7302
Karnataka	106683.5940
Tamil Nadu	173761.4066

12. Max-Min Analysis:

- Identify the district with the highest and lowest total current prices for a given year.

```
SELECT `Dist Name`, `State Name`, `Total Current Prices (in Millions Rs)`
```

```
FROM district_sectorial
```

```
WHERE Year = 2010 and `State Name` = "Madhya Pradesh"
```

```
ORDER BY "Total Current Prices (in Millions Rs)" DESC
```

```
LIMIT 1;
```

```
SELECT `Dist Name`, `State Name`, "Total Current Prices (in Millions Rs)"
```

FROM district_sectorial

WHERE Year = 2010 and `State Name` = "Madhya Pradesh"

ORDER BY "Total Current Prices (in Millions Rs)" ASC

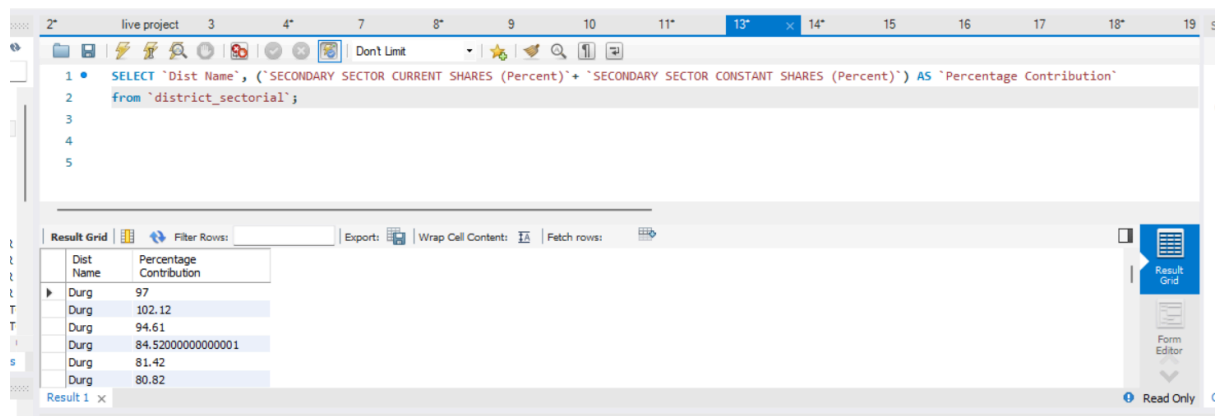
LIMIT 1;

13. Percentage Contribution:

- Calculate the percentage contribution of the secondary sector to the total economy for each district.

SELECT `Dist Name`, (`SECONDARY SECTOR CURRENT SHARES (Percent)`+ `SECONDARY SECTOR CONSTANT SHARES (Percent)`) AS `Percentage Contribution`

from `district\_sectorial`;



The screenshot shows a database query interface with a SQL editor and a results grid. The SQL query is:

```
SELECT `Dist Name`, (`SECONDARY SECTOR CURRENT SHARES (Percent)`+ `SECONDARY SECTOR CONSTANT SHARES (Percent)`) AS `Percentage Contribution`
from `district_sectorial`;
```

The results grid displays the following data:

Dist Name	Percentage Contribution
Durg	97
Durg	102.12
Durg	94.61
Durg	84.52000000000001
Durg	81.42
Durg	80.82

14. Conditional Filtering:

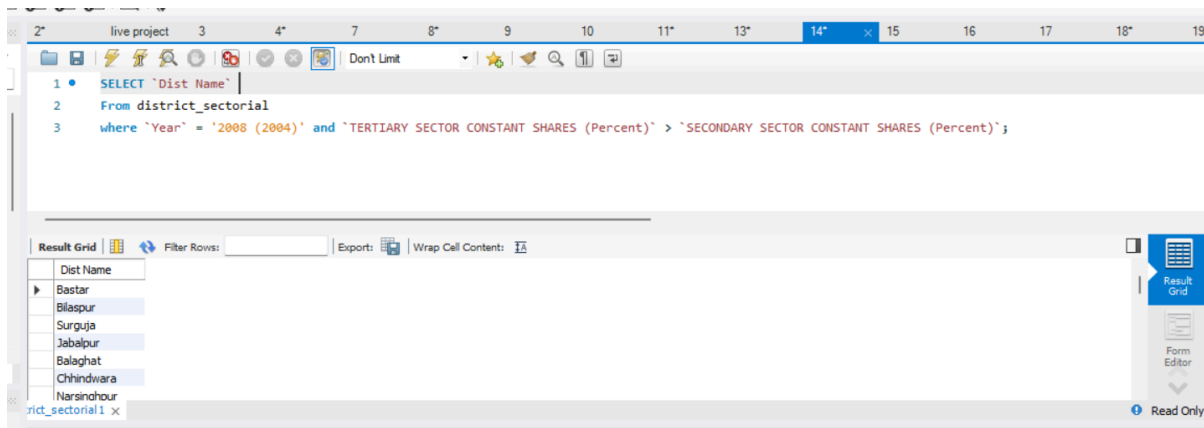
- List all districts where the tertiary sector constant share is greater than the secondary sector constant share for a specific year

SELECT `Dist Name`

From district_sectorial

where `Year` = '2008 (2004)' and `TERTIARY SECTOR CONSTANT SHARES

(Percent)` > `SECONDARY SECTOR CONSTANT SHARES (Percent)`;



15. Multi-Condition Query:

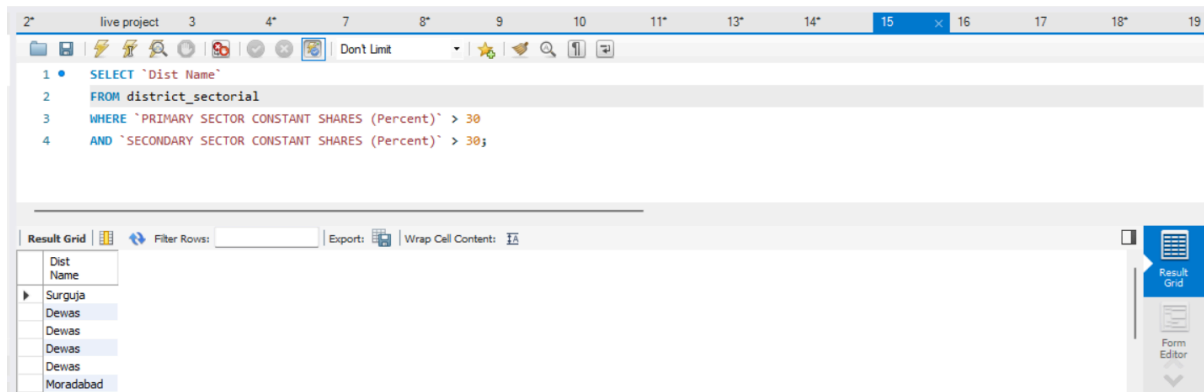
- Write a query to display districts where the primary and secondary sector shares are both greater than 30%.

SELECT `Dist Name`

FROM district_sectorial

WHERE `PRIMARY SECTOR CONSTANT SHARES (Percent)` > 30

AND `SECONDARY SECTOR CONSTANT SHARES (Percent)` > 30;



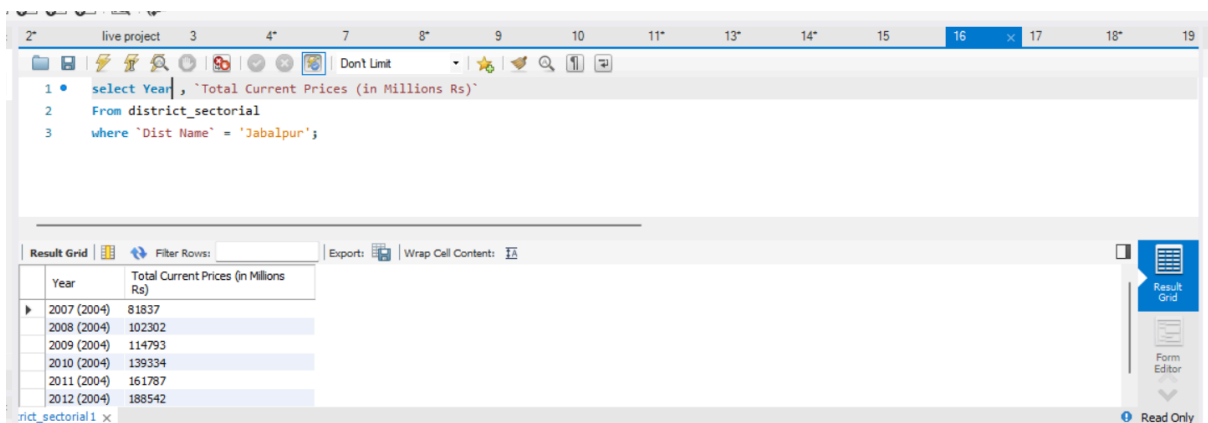
16. Time Series Analysis:

- Show how the total contribution at current prices has changed over time for a specific district.

select Year , `Total Current Prices (in Millions Rs)`

From district_sectorial

where `Dist Name` = 'Jabalpur';



17. Subquery:

- Write a subquery to find the state that contributed the most in the tertiary sector at current prices for a given year.


```

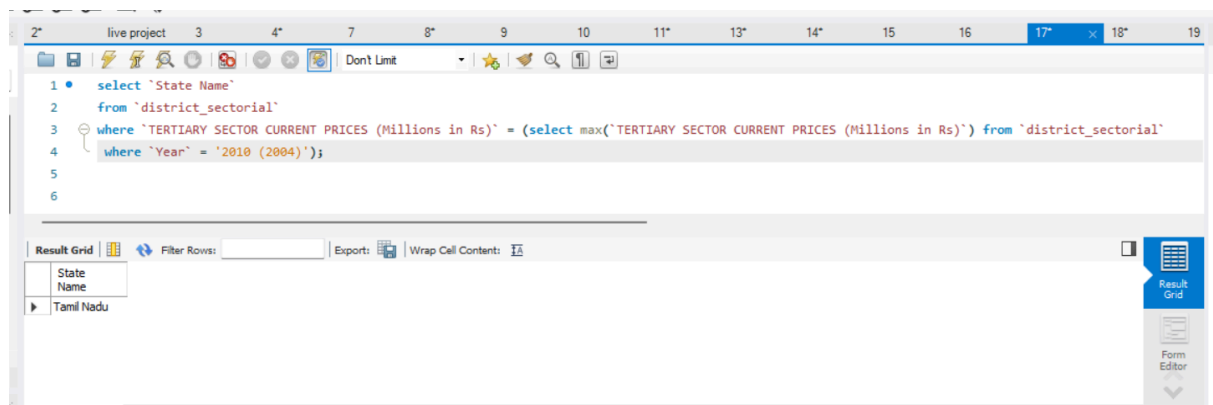
select `State Name`

from `district_sectorial`

where `TERTIARY SECTOR CURRENT PRICES (Millions in Rs)` = (select
max(`TERTIARY SECTOR CURRENT PRICES (Millions in Rs)`) from
`district_sectorial`

where `Year` = '2010 (2004)');

```



18. Distinct Query:

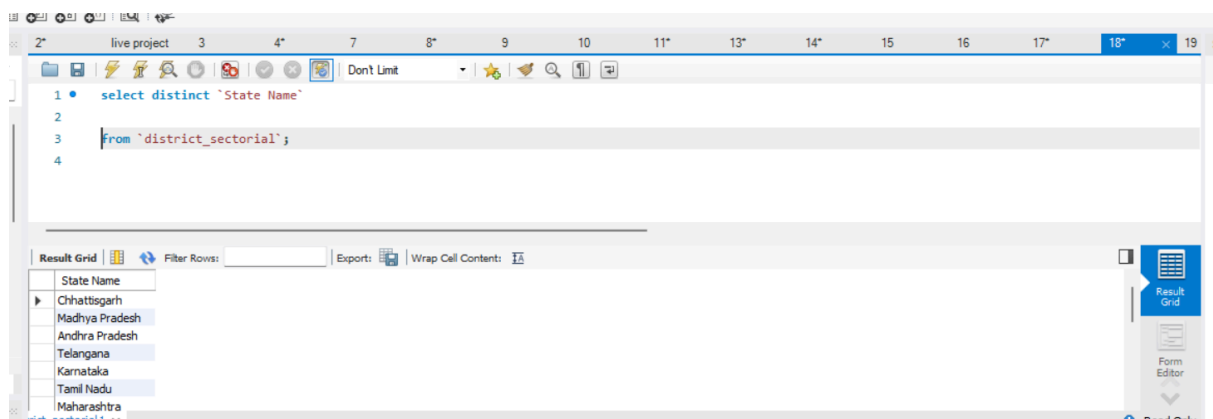
- Find all the unique states present in the dataset.

```

select distinct `State Name`

from `district_sectorial`;

```



19. Join and Aggregate:

- Join state and district data and calculate the total economic contribution for a particular state across all its districts for a given year.

SELECT `Dist Name`, SUM(`Total Current Prices (in Millions Rs)` + `Total Constant Prices (in Millions Rs)`) AS `Total Economic Contribution`

FROM `district\_sectorial`

WHERE `State Name` = 'Madhya Pradesh'

AND `Year` = '2010 (2004)'

GROUP BY `Dist Name`;

The screenshot shows a SQL IDE interface with a query editor and a results grid. The query is as follows:

```

1 SELECT `Dist Name`, SUM(`Total Current Prices (in Millions Rs)` + `Total Constant Prices (in Millions Rs)`) AS `Total Economic Contribution`
2 FROM `district_sectorial`
3 WHERE `State Name` = 'Madhya Pradesh'
4 AND `Year` = '2010 (2004)'
5 GROUP BY `Dist Name`;
6

```

The results grid displays the following data:

Dist Name	Total Economic Contribution
Jabalpur	234139
Balaghat	79098
Chhindwara	104163
Narsinghpur	58056
Seoni / Shivani	59462
Mandla	44850

20. Window Functions:

- Use a window function to calculate the cumulative total of the primary sector at constant prices for a district over the years.

select year, `PRIMARY SECTOR CONSTANT PRICES (in Millions Rs)`,

sum(`PRIMARY SECTOR CONSTANT PRICES (in Millions Rs)`) over (order by Year) as `Cumulative primary sector`

from `district\_sectorial`

where `Dist Name` = 'Jabalpur'

ct 3 4* 7 8* 9 10 11* 13* 14* 15 16 17* 18* 19* 20

1 • select year, 'PRIMARY SECTOR CONSTANT PRICES (in Millions Rs)',
2 sum('PRIMARY SECTOR CONSTANT PRICES (in Millions Rs)') over (order by Year) as 'Cumulative primary sector'
3
4 from 'district_sectorial'
5
6 where 'Dist Name' = 'Jabalpur'

Result Grid Filter Rows: Export: Wrap Cell Contents:

	year	PRIMARY SECTOR CONSTANT PRICES (in Millions Rs)	Cumulative primary sector
▶	2007 (2004)	4241	4241
	2008 (2004)	4591	8832
	2009 (2004)	4551	13383
	2010 (2004)	4936	18319
	2011 (2004)	5486	23805
	2012 (2004)	6104	29909

Result 1 x

Result Grid
 Form Editor
Read Only

Learnings

- The project reinforced knowledge of SQL, particularly in the context of complex data analysis involving large datasets. Skills in filtering, aggregation, and joining operations were honed.
- A deeper understanding of economic trends at the state and district level was gained, particularly regarding how different sectors contribute to the economy in both rural and urban regions.
- The importance of data accuracy and completeness was emphasized, as missing or inconsistent data can affect the reliability of insights.
- Exposure to data visualization tools like Power BI/Tableau enhanced the ability to present findings in a clear and visually appealing manner, making the data easier to interpret for stakeholders.

Limitations

Despite the successful completion of the project, several limitations were encountered:

1.Complex Queries

Limited SQL Capabilities: Looker Studio allows basic SQL for some data sources, but it doesn't support complex querying or advanced SQL functions as fully as dedicated BI tools like Looker itself, Power BI, or Tableau.

Calculated Fields Limitations: While Looker Studio supports calculated fields, it's not ideal for highly complex calculations or multi-step formulas. Workarounds can sometimes be created with custom SQL in data sources, but this can be cumbersome.

2.Limited Customization on Looker Studio

Chart Customization: Offers only basic charts and limited styling options, lacking advanced visualizations and conditional formatting.

Interactivity: Drill-downs and filters are limited, with no support for dynamic, multi-step filters or parameter controls for user-driven inputs.

Layout Flexibility: The grid-based layout restricts freeform design, and dashboards are not fully responsive on mobile.

Branding and Embedding: Minimal white-labeling options and limited customization for embedded reports.

3. Lack of Advanced Analytics

Limited Statistical and Predictive Functions: Looker Studio doesn't support advanced statistical analyses (like regression, clustering, or machine learning models) directly within the platform, which limits data exploration beyond basic calculations and aggregations.

Complex Calculations: Calculated fields are available but limited to simple functions. More complex multi-step calculations, custom formulas, and mathematical functions are restricted.

No Advanced Modeling: Looker Studio does not support features like predictive modeling, trend forecasting, or time series analysis, which are often required for deeper insights into data trends.

Limited Data Transformation: Data transformation within Looker Studio is basic, relying heavily on data sources to handle complex transformations or joins. It doesn't offer an ETL (Extract, Transform, Load) capability, so pre-processing needs to be done externally.

No Scripting or Automation: Unlike advanced analytics tools, Looker Studio lacks scripting (like R or Python integration), so users cannot run custom scripts or automate analytical workflows.

4. Dependency on Google Ecosystem

Limited Data Source Compatibility: Looker Studio integrates seamlessly with Google products (e.g., Google Analytics, BigQuery, Google Ads) but offers limited native connectors for other platforms. Connecting to non-Google data sources often requires third-party connectors, which may add cost or complexity.

Data Storage Reliance: The platform works best when using Google Cloud Storage or BigQuery, and using external databases may lead to performance issues. Users who prefer Amazon Web Services (AWS), Microsoft Azure, or other cloud services may find Looker Studio less compatible.

Account and Authentication: Access to Looker Studio requires a Google account, which may not align with enterprise environments that use non-Google authentication and identity management systems.

Limited Export Options: Looker Studio's export options are limited, often defaulting to Google Drive or PDF, which can be restrictive for those needing flexible export formats or direct integrations with other data systems.

5. Limited Data Transformation

Basic Data Cleaning: Looker Studio lacks robust data cleaning tools. Basic functions like text parsing, data-type conversions, or handling missing values require manual adjustments in the data source, as there's no built-in transformation layer.

Minimal Join and Blend Options: While Looker Studio allows blending data sources, the options are limited to simple, one-to-one joins. Complex joins (e.g., multiple tables or custom joins) are challenging, and there's limited flexibility in merging disparate data sources without pre-processing.

Calculated Fields Limitations: Calculated fields in Looker Studio support only basic math, string functions, and aggregations. Advanced calculations, such as complex conditional logic or nested formulas, are difficult to achieve and may require transformation outside Looker Studio.

No ETL (Extract, Transform, Load) Capabilities: Looker Studio lacks ETL functions, so data must be fully prepped in external tools or databases before import. Users needing

transformations like pivoting, aggregating, or filtering at a granular level need to rely on their data source.

Lacks powerful data transformation capabilities.

Conclusions

The "District-Wise Sectoral Analysis of Indian States" project provided valuable insights into the economic landscape of India's districts across different sectors. Through SQL queries, key trends and sectoral growth patterns were identified, such as the dominance of the tertiary sector in urban areas and the primary sector's relative importance in rural districts. The analysis revealed disparities in growth rates between districts, highlighting opportunities for targeted economic interventions. This project also underscored the importance of data quality in economic analysis and the potential for further research using more complete datasets.

DashBoard

here is the link of dashboard that we have made based on the given data -

<https://lookerstudio.google.com/reporting/51736145-1f22-4270-a5be-73a392515cde>